UNIT IV – MULTIMODAL GENERATIVE AI APPLICATIONS

QUESTION 2

ENGINEERING-SUPPORT CHATBOT USING NLP

Question: Design an engineering-support chatbot that can answer technical questions and provide relevant solutions using NLP techniques.

This implementation develops a Streamlit-based engineering-support chatbot that accepts a student's technical question, preprocesses the text, represents the question using TF-IDF, calculates Cosine Similarity against an engineering knowledge base, identifies the most relevant topic, and displays the corresponding technical solution.

1. AIM

To design and implement an engineering-support chatbot that uses Natural Language Processing (NLP) techniques to understand technical questions and provide relevant engineering explanations and solutions.

2. OBJECTIVES

- Accept technical questions from engineering students through a user-friendly interface.
- Preprocess the input text using basic NLP techniques.
- Convert text into numerical representations using TF-IDF vectorization.
- Measure the similarity between the user's question and engineering knowledge using Cosine Similarity.
- Identify the most relevant engineering topic.
- Provide a clear and relevant technical solution.
- Display the result through an interactive Streamlit web application.

3. SOFTWARE AND HARDWARE REQUIREMENTS

- Programming Language: Python
- Web Framework: Streamlit
- NLP Library: Scikit-learn
- NLP Techniques: Text preprocessing, TF-IDF and Cosine Similarity
- Development Environment: Visual Studio Code / Python terminal
- Operating System: Windows or any system supporting Python
- Hardware: Standard computer with sufficient RAM and storage

4. SYSTEM WORKFLOW

Student Technical Question
↓
Text Preprocessing
↓
TF-IDF Vectorization
↓
Cosine Similarity Calculation
↓
Relevant Engineering Topic
↓
Technical Knowledge Retrieval
↓
Engineering Solution Display

Figure 5. NLP workflow of the Engineering-Support Chatbot.

5. NLP TECHNIQUES USED

- Text Preprocessing: Converts the input into lowercase text, removes unnecessary characters and normalizes spaces.
- TF-IDF Vectorization: Converts textual documents and the user's question into numerical feature vectors.
- Cosine Similarity: Measures how closely the question matches each engineering topic in the knowledge base.
- Knowledge-Based Retrieval: Selects the topic with the highest similarity score and displays its stored technical explanation.

6. STEP-BY-STEP IMPLEMENTATION

Step 1 – Create the project folder: Create a folder named Q2_Engineering_Support_Chatbot inside UNIT_IV_Multimodal_Generative_AI.

Step 2 – Create the Python file: Create the file engineering_support.py inside the Q2_Engineering_Support_Chatbot folder.

Step 3 – Install the required library: Activate the Python virtual environment and install Scikit-learn using: pip install scikit-learn

Step 4 – Import libraries: Import Streamlit, regular expressions, TfidfVectorizer and cosine_similarity.

Step 5 – Prepare the engineering knowledge base: Create a small knowledge base containing topics such as Ohm's Law, Newton's Second Law, Binary Search, Operating Systems, Database Normalization and Computer Networks.

Step 6 – Preprocess the text: Convert questions to lowercase, remove unnecessary characters and normalize whitespace.

Step 7 – Apply TF-IDF: Convert the engineering knowledge base into TF-IDF vectors.

Step 8 – Calculate Cosine Similarity: Transform the user's question into a TF-IDF vector and compare it with all knowledge-base vectors.

Step 9 – Select the relevant topic: Choose the topic having the highest similarity score.

Step 10 – Display the solution: Display the retrieved engineering explanation, relevant topic and NLP similarity score in the Streamlit interface.

Step 11 – Run the application: From the main project directory, run: streamlit run ".\Q2_Engineering_Support_Chatbot\engineering_support.py"

Step 12 – Test the chatbot: Test the application using technical questions such as 'What is Ohm's law?', 'Explain binary search', 'What is Newton's second law?' and 'What is database normalization?'.

7. ALGORITHM

1. Start the chatbot.
2. Load the engineering knowledge base.
3. Preprocess the knowledge-base text.
4. Generate TF-IDF vectors for the knowledge base.
5. Accept the student's engineering question.
6. Preprocess the question.
7. Convert the question into a TF-IDF vector.
8. Calculate Cosine Similarity between the question and all knowledge-base entries.
9. Find the entry with the highest similarity score.
10. Display the relevant engineering topic and technical solution.
11. Stop.

8. COMPLETE IMPLEMENTATION CODE

```
import streamlit as st
import re
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

st.set_page_config(
    page_title="Engineering Support Chatbot",
    page_icon="🛠",
    layout="centered"
)

st.title("🛠 Engineering Support Chatbot")
st.write(
    "Ask technical engineering questions and receive "
    "relevant explanations and solutions using NLP techniques."
)

knowledge_base = [
    {
        "topic": "Ohm's Law",
        "keywords": "ohm law voltage current resistance electrical circuit",
```

```
    "answer": (
        "Ohm's Law defines the relationship between voltage, "
        "current, and resistance in an electrical circuit. "
        "The formula is V = I × R, where V is voltage, "
        "I is current, and R is resistance."
    )
},
{
    "topic": "Newton's Second Law",
    "keywords": "newton second law force mass acceleration mechanics",
    "answer": (
        "Newton's Second Law states that the force acting on "
        "an object is equal to its mass multiplied by its "
        "acceleration. The formula is F = m × a."
    )
},
{
    "topic": "Kirchhoff's Current Law",
    "keywords": "kirchhoff current law kcl circuit node current",
    "answer": (
        "Kirchhoff's Current Law states that the total current "
        "entering a circuit node is equal to the total current "
        "leaving that node. It is based on conservation of charge."
    )
},
{
    "topic": "Kirchhoff's Voltage Law",
    "keywords": "kirchhoff voltage law kvl circuit loop voltage",
    "answer": (
        "Kirchhoff's Voltage Law states that the algebraic sum "
        "of all voltages around a closed circuit loop is zero. "
        "It is based on conservation of energy."
    )
},
{
    "topic": "Binary Search",
    "keywords": "binary search algorithm sorted array data structure",
    "answer": (
        "Binary Search is a searching algorithm used on sorted "
        "data. It repeatedly divides the search range into two "
        "halves. Its time complexity is O(log n)."
    )
},
{
    "topic": "Operating System",
    "keywords": "operating system os process memory file linux windows",
    "answer": (
        "An Operating System is system software that manages "
        "computer hardware and software resources. It manages "
        "processes, memory, files, input/output devices, and "
        "provides an interface for users and applications."
    )
```

```
    },
    {
        "topic": "Database Normalization",
        "keywords": "database normalization 1nf 2nf 3nf sql tables redundancy",
        "answer": (
            "Database normalization is the process of organizing "
            "data into related tables to reduce data redundancy "
            "and improve data integrity. Common normal forms are "
            "1NF, 2NF, and 3NF."
        )
    },
    {
        "topic": "Computer Networks",
        "keywords": "computer network ip router switch tcp udp communication",
        "answer": (
            "A computer network connects multiple devices so they "
            "can communicate and share resources. Important "
            "components include IP addresses, routers, switches, "
            "TCP, and UDP."
        )
    },
    {
        "topic": "Python Programming",
        "keywords": "python programming language variables loops functions code",
        "answer": (
            "Python is a high-level programming language known for "
            "its simple syntax. It is widely used for software "
            "development, data science, artificial intelligence, "
            "automation, and machine learning."
        )
    },
    {
        "topic": "Machine Learning",
        "keywords": "machine learning artificial intelligence supervised unsupervised prediction",
        "answer": (
            "Machine Learning is a branch of artificial intelligence "
            "that enables computers to learn patterns from data and "
            "make predictions or decisions. Major types include "
            "supervised, unsupervised, and reinforcement learning."
        )
    }
]

def preprocess_text(text):
    text = text.lower()
    text = re.sub(r"[^a-zA-Z0-9\s]", "", text)
    text = re.sub(r"\s+", " ", text)
    return text.strip()

documents = []

for item in knowledge_base:
```

```
    document = (
        item["topic"] + " "
        + item["keywords"] + " "
        + item["answer"]
    )
    documents.append(preprocess_text(document))

vectorizer = TfidfVectorizer(stop_words="english")
tfidf_matrix = vectorizer.fit_transform(documents)

def find_solution(question):
    processed_question = preprocess_text(question)
    question_vector = vectorizer.transform([processed_question])
    similarity_scores = cosine_similarity(
        question_vector, tfidf_matrix
    )[0]
    best_index = similarity_scores.argmax()
    best_score = similarity_scores[best_index]
    return knowledge_base[best_index], best_score

question = st.text_area(
    "Enter your technical engineering question:",
    placeholder="Example: What is Ohm's law?"
)

if st.button("🔧 Get Technical Solution"):
    if question.strip() == "":
        st.warning("Please enter a technical question.")
    else:
        topic, score = find_solution(question)

        if score >= 0.10:
            st.subheader("🧠 Engineering Solution")
            st.success(topic["answer"])
            st.info(f"Relevant Topic: {topic['topic']}")
            st.write(f"NLP Similarity Score: {score:.2f}")
        else:
            st.warning(
                "The question is not sufficiently related to "
                "the available engineering knowledge base."
            )

st.divider()
st.subheader("💡 Example Technical Questions")

examples = [
    "What is Ohm's law?",
    "Explain Newton's second law.",
    "What is binary search?",
    "Explain Kirchhoff's Current Law.",
    "What is database normalization?",
```

```
    "What is an operating system?",
    "Explain computer networks.",
    "What is machine learning?"
]

for example in examples:
    st.write("• " + example)

st.divider()
st.caption(
    "Engineering Support Chatbot | "
    "NLP using TF-IDF and Cosine Similarity"
)
```

9. INPUT AND OUTPUT

Input:

- What is Ohm's law?
- Explain binary search.
- What is Newton's second law?
- What is database normalization?

Output:

- The chatbot identifies the most relevant engineering topic using TF-IDF and Cosine Similarity.
- It displays the corresponding technical explanation and the NLP similarity score.

10. IMPLEMENTATION RESULTS AND SCREENSHOTS

The following screenshots show the successfully implemented chatbot and its responses to different engineering questions. They provide evidence that the NLP-based retrieval system identifies relevant topics and returns appropriate technical solutions.

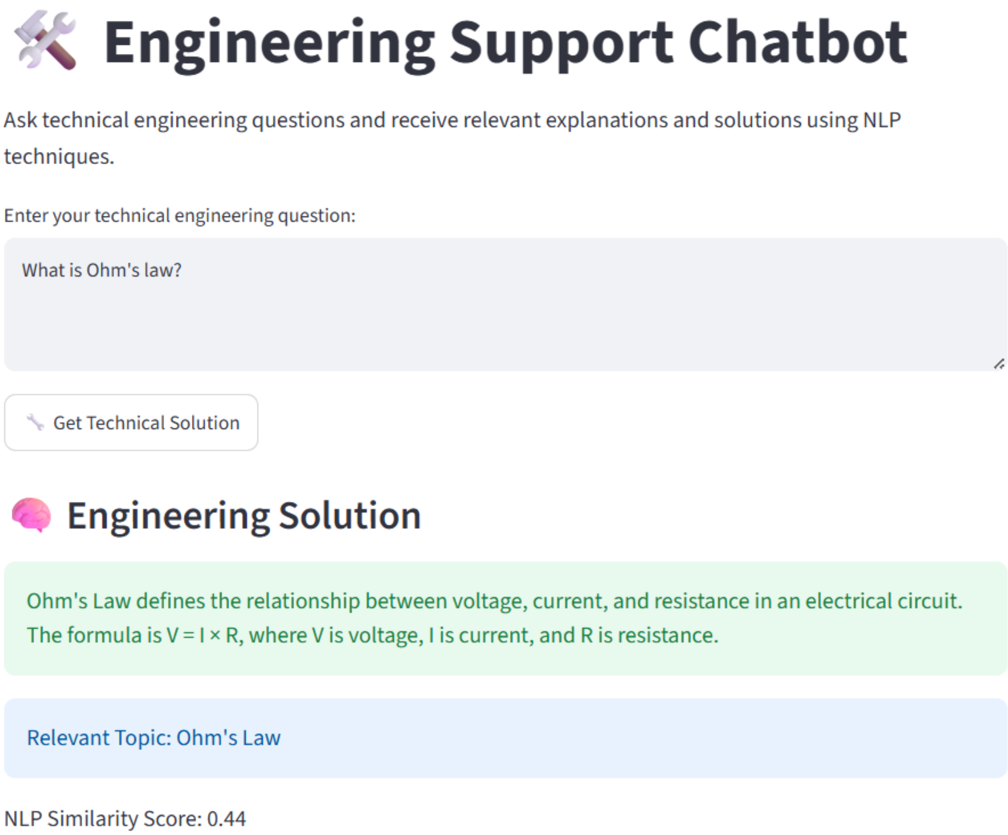

Figure 1. Engineering Support Chatbot – Ohm's Law query and NLP result.

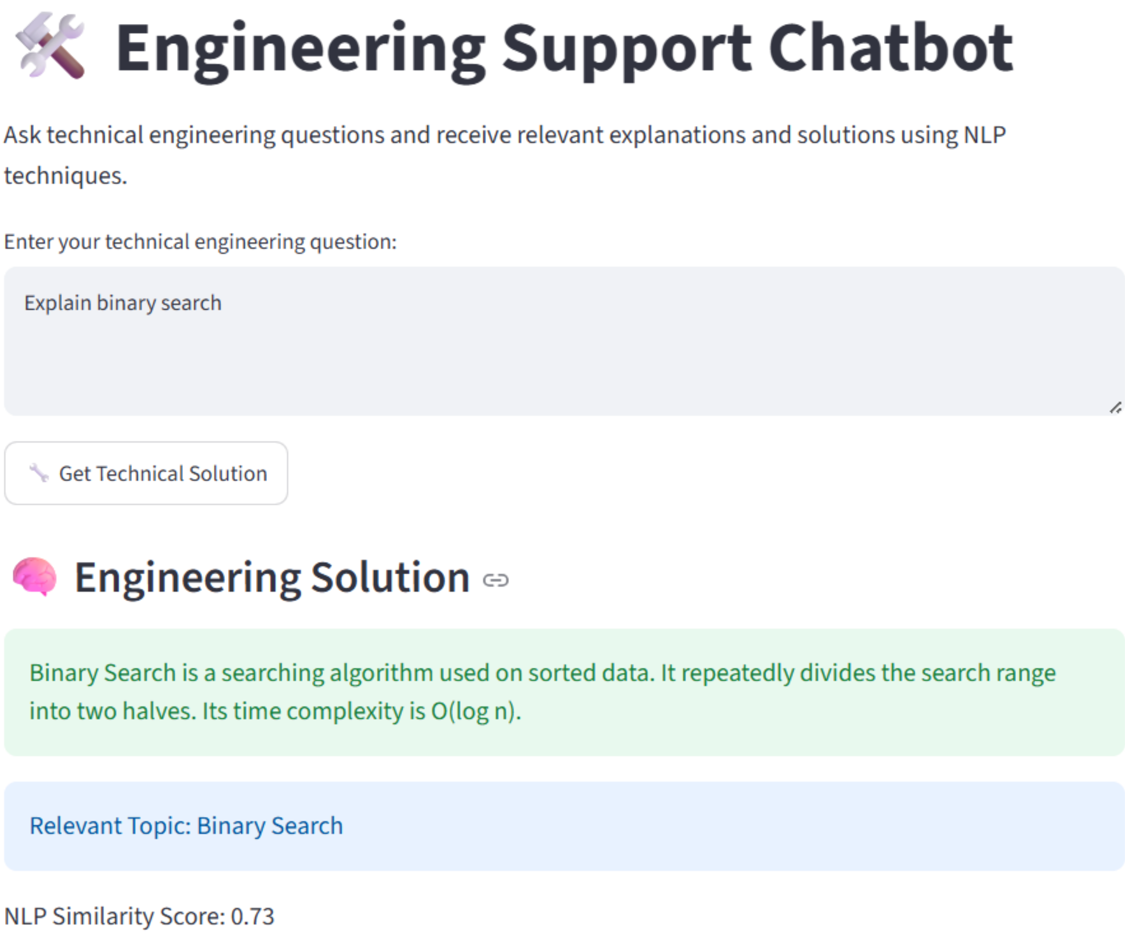

Figure 2. Engineering Support Chatbot – Binary Search query and NLP result.

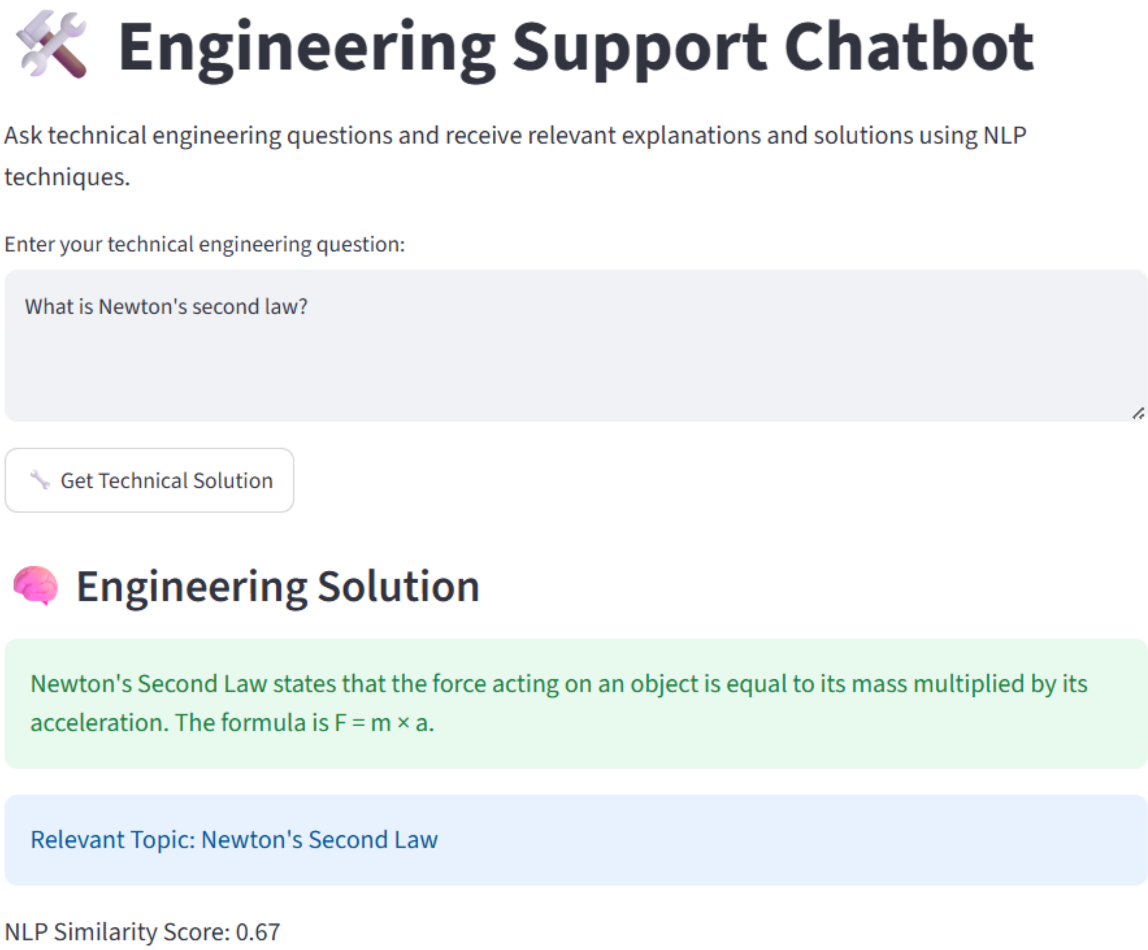

Figure 3. Engineering Support Chatbot – Newton's Second Law query and NLP result.

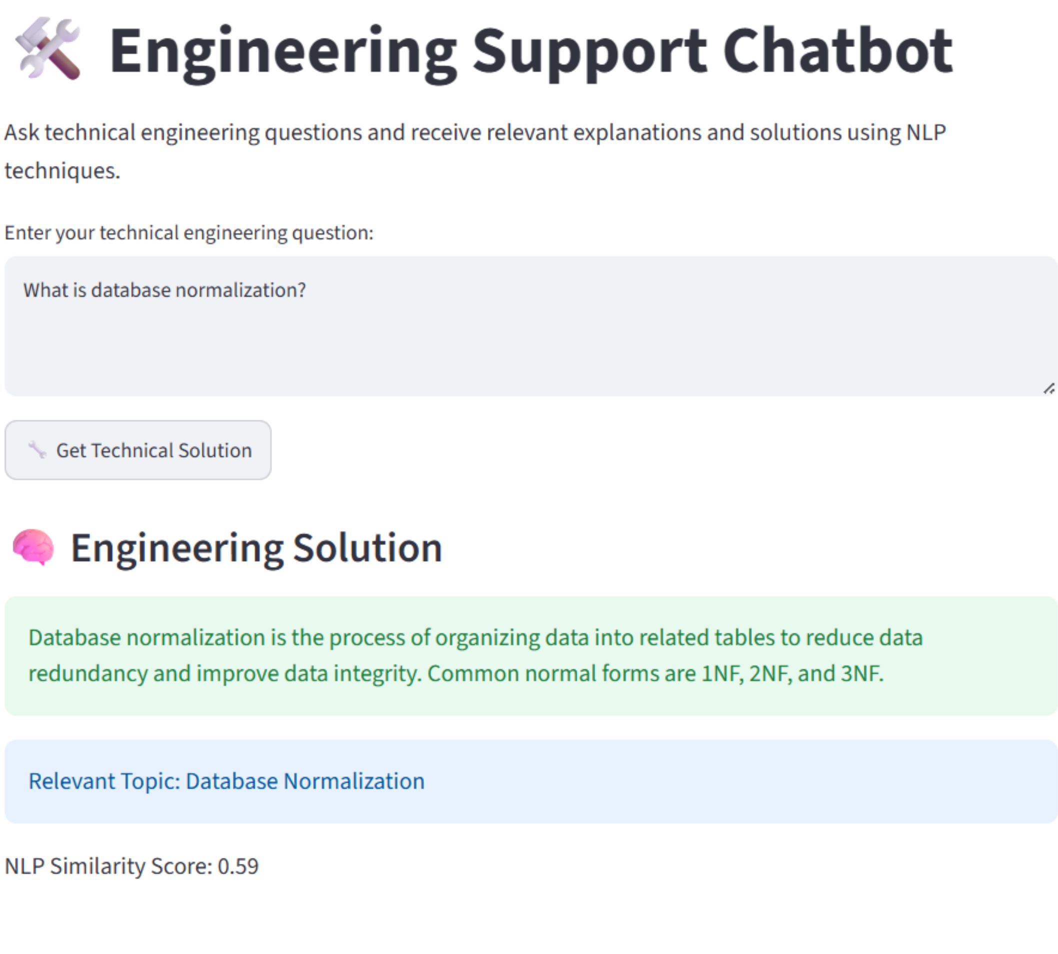

Figure 4. Engineering Support Chatbot – Database Normalization query and NLP result.

11. TEST RESULTS

Test Case	Input Query	Detected Topic	Similarity Score
1	What is Ohm's law?	Ohm's Law	0.44
2	Explain binary search	Binary Search	0.73
3	What is Newton's second law?	Newton's Second Law	0.67
4	What is database normalization?	Database Normalization	0.59

12. OBSERVATION

- The chatbot successfully accepts engineering-related natural language questions.
- TF-IDF represents the question and knowledge-base content as numerical vectors.
- Cosine Similarity identifies the most relevant engineering topic.
- The tested questions produced relevant technical answers.
- The similarity score provides an indication of how closely the question matches the available knowledge.

13. ADVANTAGES

- Simple and easy-to-use interface.
- Uses standard NLP techniques that are easy to understand and demonstrate.
- Fast response because the system works with a small local knowledge base.
- Does not require training a new machine-learning model.
- Can be extended by adding more engineering topics and solutions.

14. LIMITATIONS

- The chatbot can provide answers only for topics represented in its knowledge base.
- Very complex or ambiguous technical questions may not receive the ideal response.
- The similarity score depends on the words present in the question and knowledge base.
- The current implementation is a retrieval-based NLP chatbot rather than a fully generative engineering expert.

15. FUTURE ENHANCEMENTS

- Add a larger engineering knowledge base covering multiple branches.
- Integrate a pre-trained generative language model when a suitable local or API-based model is available.
- Add speech input and text-to-speech output.
- Add document-based question answering using engineering notes and PDFs.
- Add multilingual support for Indian languages.

16. RESULT

The Engineering-Support Chatbot was successfully designed and implemented using NLP techniques. The system preprocesses the student's question, applies TF-IDF vectorization, uses Cosine Similarity to identify the most relevant engineering topic, and displays the corresponding technical solution. The test results demonstrate that the chatbot can effectively support common engineering-related technical queries.

17. CONCLUSION

The developed chatbot demonstrates a practical application of Natural Language Processing in engineering education. By combining text preprocessing, TF-IDF and Cosine Similarity, the system can match student questions with relevant technical knowledge and provide clear answers through an interactive Streamlit interface. The implementation is simple, fast and suitable for an academic demonstration, while also providing a foundation for future expansion into a more advanced AI-based engineering assistant.

18. VIVA / KEY POINTS

- NLP stands for Natural Language Processing.
- TF-IDF stands for Term Frequency–Inverse Document Frequency.
- Cosine Similarity measures similarity between two vectors.
- Streamlit is used to create the interactive web interface.

- The chatbot uses a knowledge-based retrieval approach.
- The system can be extended by adding more engineering topics.